Database Normalization

06 October 2026 18:57

Normalization is a systematic technique used in relational database design to organize data into well-structured tables.

The main goal are to:

- Reduce data redundancy.
- Prevent insertion anomalies.
- Prevent update anomalies.
- Prevent deletion anomalies.
- Improve data integrity.
- To make dependencies between Data clear.
- Produce a table that are easier to maintain.

The normal forms are usually studied in this order:

UNF
|
v
1NF
|
v
2NF
|
v
3NF
|
v
BCNF
|
v
4NF
|
v
5NF

Each normal form imposes additional condition on the database design.

Why Normalization is Required?

Consider this table:

Student_id	Name	Course_id	Course_name	Instructor
101	Alice Johnson	C101	Database Management	Dr. Smith
101	Alice Johnson	C102	Data Structures	Prof. Brown
101	Alice Johnson	C103	Computer Networks	Dr. Wilson
102	Bob Williams	C101	Database Management	Dr. Smith
102	Bob Williams	C104	Operating Systems	Prof. Taylor
103	Charlie Davis	C105	Software Engineering	Dr. Anderson
103	Charlie Davis	C106	Web Development	Prof. Thomas
104	Diana Miller	C102	Data Structures	Prof. Brown
105	Ethan Clark	C107	Machine Learning	Dr. Garcia
105	Ethan Clark	C108	Cloud Computing	Prof. Martin

There is redundancy.
For example:
Student_id = 101
Name = Alice Johnson
Is repeated.
Similarly:
C101 -> Database Management->Dr.Smith
is repeated for every student taking C101.

C102 ->Data Structures -> Prof.Brown
is repeated for every student taking C102.

This can produce several problems.

Objectives of Normalization

- **Reduce Data redundancy:** Avoid storing the same information repeatedly.
- **Prevent insertion anomalies:** Allow new information to be inserted without requiring unrelated information.
- **Prevent update anomalies:** Ensure a change does not need to be made in multiple places.
- **Prevent deletion anomalies:** Avoid accidentally losing useful information when deleting a record.
- **Improve data integrity:** Keep relationship and facts consistent.

Understanding database dependencies

1. **Functional dependency(FD):** Of functional dependency exists when the value of one attribute uniquely determines the value of another attribute.
   It is represented as:

   X -> Y

   Here X is determinant and Y is the dependent attribute.

   For example, consider a Student table:

StudentID	StudentName	Department
101	Amit	CSE
102	Neha	Commerce
103	Ravi	CSE

   Assuming each StudentID identifies exactly one student.

   StudentID -> StudentName

   StudentID -> Department

   This means that knowing the StudentID determines the students name and department.
   However, the reverse is not necessarily true.

StudentName -> StudentID

Because two students may have the same name.

Types of dependencies

1. **Functional Dependency:** One attribute or group of attributes determines another attribute.
   Example: StudentID -> StudentName

2. **Full functional dependency:** An attribute depends on the entire composite key, not just a part of it. Example: (StudentID, CourseID) -> Grade, where neither StudentID alone nor CourseID alone determines the grade for a particular student-course enrollment.

3. **Partial dependency:** A non-key attribute depends on only part of a composite candidate key.
   Example: (StudentID, CourseID) -> StudentName but actually StudentID -> StudentName.

   Consider Employee_Project table

Employee_ID	Project_ID	Employee_Name	Project_Name	Hours_Worked
E101	P01	Alice	AI Alpha	20
E101	P02	Alice	Cloud Beta	15
E102	P01	Harry	AI Alpha	30

   Composite Primary Key: {Employee_ID, Project_ID)
   Non-Prime Attributes: Employee_Name, Project_Name, Hours_Worked

   Identifying the Partial Dependencies

   1) **Employee_ID -> Employee_Name**
      Issue: You only need Employee_ID to know employee's name. You don't need Project_ID. **This is partial dependency.**

   2) **Project_ID -> Project_Name**
      Issue: You only need Project_Id to know project name. You don't need Employee_ID. **This is partial dependency.**

   3) **{Employee_ID, Project_ID} -> Hours_worked**
      Valid: Ours worked depends on both who the employee is and which project they are working on. This is a **full functional dependency.**

4. **Transitive dependency:** An attribute depends on a key indirectly through another non - key attribute. Example: StudentID -> DeptID and DeptID-> DeptName; therefore, StudentID -> DeptName

5. **Multivalued dependency(MVD):** One attribute determines a set of values for another attribute, independently of other attributes. It is represented as X ->> Y. Example: A student may have multiple independent hobbies and multiple independent languages.

Another example of Multi-valued Dependency

Consider a table Student

StudentID(X)	Course(Y)	Hobby(Z)
101	Computer Science	Music
101	Computer Science	Sports
101	Mathematics	Music
101	Mathematics	Sports

Why is this a Multivalued Dependency?
For student _ID. 101 There is a set of courses. {Computer Science, Mathematics } And a set of hobbies. {music, sports.} Because these two sets are completely independent, the database is forced to repeat every combination of course and hobby for that student.

If student 101 decides to pick up a new hobby like painting, we cannot just add one row. We would have to insert two new rows to pair painting with both computer science and mathematics.

This gives us two MVDs:
i. Student_ID ->> Course
ii. Student_ID ->> Hobby

Keys used in normalization

- **SuperKey**: Any attribute or combination of attributes that uniquely identifies a row.
- **Candidate Key:** A minimal super key. Removing any attribute makes it no longer a super key.
- **Primary Key:** The candidate key selected to identify rows.
- **Prime Attribute:** An attribute that belongs to at least one candidate key.
- **Non-Prime Attribute:** An attribute that does not belongs to any candidate key.

First Normal Form (1NF)

A table is in first normal form if each cell contains a single value from the relevant domain with no repeating groups or multi-valued field stored as a single attribute.

In simple terms, each row - column intersection should contain one value, not a list of values.

Example: Table not in 1NF

StudentID	StudentName	MobileNumber
101	Amit	9876500000, 7890654321
102	Neha	6789098765

The MobileNumber column contains multiple values in a single cell

Covert it into 1NF
In this step we removed the multi-valued attribute.

StudentId	StudentName	MobileNumber
101	Amit	9876500000
101	Amit	7890654321
102	Neha	6789098765

Now each cell contains one mobile number.

An alternative design, often better for database application, is to separate the phone numbers into a related table.

Student

StudentID(PK)	StudentName
101	Amit
102	Neha

StudentMobile

StudentID(FK)	MobileNumber
101	9876500000
101	7890654321
102	6789098765

Here StudentID in StudentMobile references the Student table.

Key rule for 1NF: Eliminate repeating groups and ensure every field contains a single value.

Second Normal Form(2NF)

A table is in second normal form if:

i. It is already in 1NF.
ii. Every non - prime attribute is fully functionally dependent on every candidate key. In the common textbook formulation, for a single composite primary key, this means eliminating partial dependencies on parts of that key.

Example: Table not in 2NF

Consider the following table:

Enrollment:

StudentID	CourseID	StudentName	CourseName	Grade
101	C01	Amit	Java	A
101	C02	Amit	Python	B
102	C01	Neha	Java	A

Suppose the composite primary key is:

(StudentID,CourseID)

The functional dependencies are:

StudentID -> StudentName

CourseID -> CourseName

(StudentID, CourseID) -> Grade

The problem is that StudentName depends on StudentID and CourseName depends only on CourseID. They do not depend on entire composite key.

These are partial dependencies, so the table violates 2NF.

Convert it into 2NF
Decompose the table into three tables.

Student

StudentID	StudentName
101	Amit
102	Neha

Course

CourseID	CourseName
C01	Java
C02	Python

Enrollment

StudentID(PK,FK)	CourseID(PK,FK)	Grade
101	C01	A
101	C02	B
102	C01	A

Now:

- StudentName depends on StudentID in the Student table.
- CourseName depends on CourseID in the Course table.
- Grade depends on the combination of StudentID and CourseID in the Enrollment table.

Key rule for 2NF: Remove partial dependencies of non - prime attributes on candidate keys.

Note: If every candidate key consist of a single attribute, the table already in 1NF is automatically into 2NF.

Third Normal Form (3NF)

A table is in 3rd normal form if:

1. It is already in 2NF.
2. For every non - trivial functional dependency X -> A, either X is a super key or A is a prime attribute.

A useful introductory way to understand three NF is: Eliminate transitive

dependencies of non-key attributes on a key.

Example: Table not in 3NF
Consider the following table:

Student

StudentID(PK)	StudentName	DeptID	DeptName
101	Amit	D01	Computer Science
102	Neha	D02	Commerce
103	Ravi	D01	Computer Science

Assume each department has exactly one department name.

The functional dependencies are:

StudentID -> StudentName, DeptID

DeptID -> DeptName

Therefore:
StudentID -> DeptName

Indirectly through DeptID

This is a transitive dependency:

StudentID -> DeptID -> DeptName

DeptName Describes the department, not the student. Storing it repeatedly causes redundancy.

Convert it into 3NF
Separate student and department information.

Student

StudentID(PK)	StudentName	DeptID(FK)
101	Amit	D01
102	Neha	D02
103	Ravi	D01

Department

DeptID(PK)	DeptName
D01	Computer Science
D02	Commerce

Now the department name is stored only one per department.

Why this is better?
Suppose the department name changes from computer science to computer science and engineering.

- In the original table, several student rows might need updating.

- In the normalized design, only one row in the department table needs updating.

Key rule for 3NF: Remove inappropriate transitive dependencies while preserving the required functional relationship.

Important distinction
Strictly speaking, 3NF is slightly more general than simply saying "no transitive dependencies." Its formal definition allows certain dependencies when the dependent attribute is prime. This distinctions becomes important when studying BCNF.

Boyce-Codd normal form (BCNF)
BCNF is a stronger version of 3NF.
A relation is in Boyce-Codd normal form if, for every non - trivial functional dependency:

X -> Y
X must be a super key

In simple terms, **every determinant must be a super key.**

Example: A table in 3NF but not in BCNF

Consider a University where:
- Each student can enroll in multiple courses.
- Each instructor teaches only one course.
- A course can be taught by multiple instructors.
- Each student has one instructor per course.

Consider this relation

StudentInstructor

StudentID	CourseID	Instructor
S01	C01	I01
S02	C01	I01
S01	C02	I02
S03	C02	I02

Assume the following dependencies:

(StudentID, CourseID) -> Instructor

Instructor -> CourseID

The first dependency means a student and course together determine the instructor. The second means an instructor teaches only 1 course.
The candidate keys are:
- (StudentID, CourseID)
- (StudentID, Instructor)

The attribute CourseID is a prime attribute because it belongs to a candidate key.
However:

Instructor -> CourseID

Here Instructor is not a super key because an instructor can teach multiple students. Therefore, the relation violates BCNF.

It can still satisfy 3rd normal form because CourseID is a prime attribute.

Convert it into BCNF
Decompose the original relation into two tables.

InstructorCourse

Instructor(PK)	CourseID
I01	C01
I02	C02

StudentInstructor

StudentID	Instructor
S01	I01
S02	I01
S01	I02
S03	I02

In the first table, Instructor determines CourseID. In the second table the student-instructor relationship is recorded.

The decomposition is lossless under the stated dependency assumption. Joining the two tables on instructor reconstructs the original relation.

Key rule for BCNF: Every determinant of a non - trivial functional dependency must be a super-key.

3NF versus BCNF

Feature	**3NF**	**BCNF**
Strength	**Less restrictive**	**More restrictive**
Determinant must be a Super key	**Not always**	**Always**
May allow a dependency where the RHS is prime.	**Yes**	**No, unless the Determinant is a super key.**
Redundancy caused by certain overlapping candidate keys.	**May remain.**	**Most likely to be eliminated.**
Dependency preservation after decomposition.	**Often achievable.**	**Not always achievable.**

BCNF decomposition can sometimes fail to preserve all functional dependencies directly within the decomposed table, even though it is lossless. This is an important trade off in database design.

Fourth Normal Form (4NF)

4th normal form addresses a different problem: **multi-valued dependencies.**

X ->> Y
X is a super-key.

A multi-valued dependency occur when one attribute determines a set of values for another attribute independently of the remaining attributes.

Example: Table not in 4NF
Suppose a student can have multiple hobbies and speak multiple languages. Assume hobbies and languages are independent of each other for each student.

StudentDetails

StudentID	Hobby	Language
101	Cricket	English
101	Cricket	Hindi
101	Music	English
101	Music	Hindi

Student 101 has two hobbies and two languages.
Because the hobbies and. Languages are independent. Every hobby is paired with every language for that student.

The multi value dependencies are:

StudentID ->> Hobby

StudentID ->> Language

The candidate key of the original table is (StudentID, Hobby, Language)
StudentID alone is not a super key. So the non - trivial multi-valued dependencies violate 4NF.

This creates unnecessary combination. If the student has three hobbies and 4 languages, the table may need 12 combination for just one student.

Convert it into 4NF
Split the independent multi valued facts into separate tables.

StudentHobby

StudentID	Hobby
101	Cricket
101	Music

StudentLanguage

StudentID	Language
101	English
101	Hindi

Now hobbies and languages are stored independently without generating every possible combination.

Key rule 4NF: Separate independent multi - valued relationship into different tables when they are multi-valued dependencies violate 4NF.

The independence assumption matters. If a particular hobby - language pairing itself has meaning, the original combinations may represent real data and should not automatically be discarded.

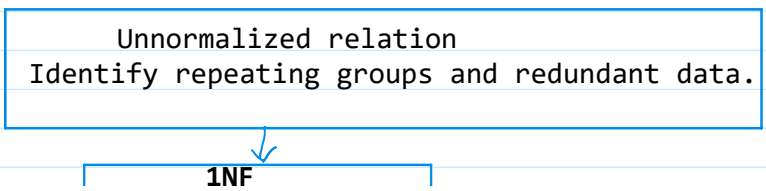
Summary of all normal forms

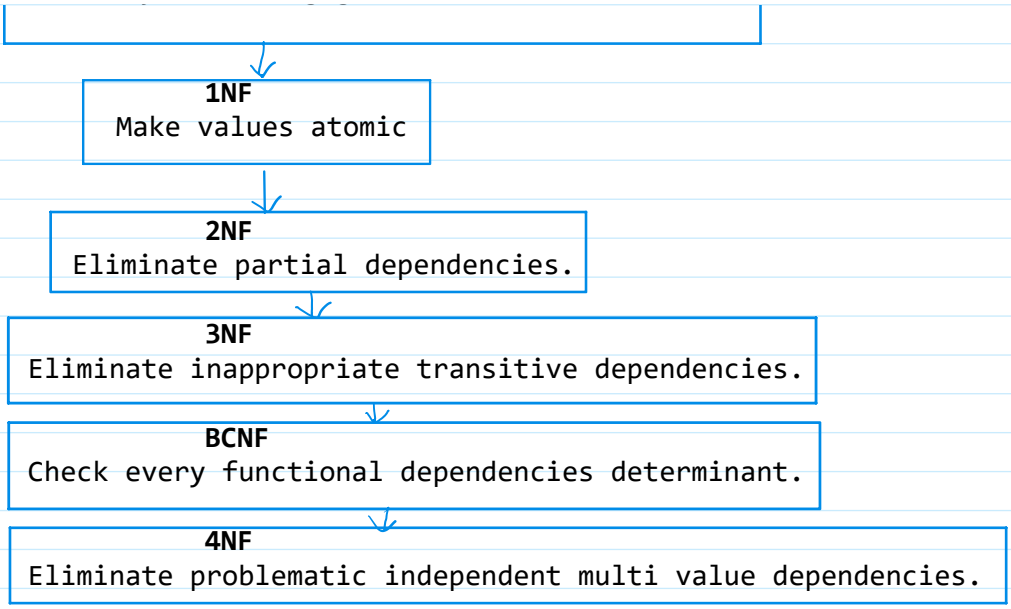
1NF	**First Normal Form:** Every cell contains a single value, eliminate repeating groups. Problem addressed: multi-valued cell and repeating groups.
2NF	**Second Normal Form:** Be in 1NF and eliminate partial dependencies of non-prime attribute on candidate keys. Problem addressed: Attributes depending on only part of a composite key.
3NF	**Third Normal Form:** Be in 2NF and satisfy the formal 3NF functional dependency condition. Problem addressed: Many transitive dependencies and redundant facts.
BCNF	**Boyce-Codd Normal Form:** Every determinant of a non-trivial functional dependency must be a super key. Problem addressed: Certain redundancies that remain in three NF.
4NF	**Fourth Normal Form:** Every determinant of a non - trivial multi - valued dependency must be a super key. Problem addressed: Independent multi-valued facts stored together.

Which dependency causes which problem?

Concept	**Example**	**Normal form Concerned**
Repeating groups	One cell stores several mobile numbers.	**1NF**
Partial dependency.	StudentID -> StudentName when the key is (StudentID, CourseID)	**2NF**
Transitive dependency	StudentID -> DeptID -> DeptName	**3NF**
Determinant is not a Super-key	Instructor -> CourseID	**BCNF**
Independent multi-valued Facts	StudentID->Hobby and StudentID-> Language	**4NF**

How normalization is applied in database design?





In practice many database application aim for 3NF or BCNF or 4NF is useful when independent multi valued relationships create redundancy. A higher normal form is not automatically better in every situation. Designers must also consider constraints, query requirements, dependency preservations, and performance.